



기본 관계형 데이터 개념

관계형 데이터 이해

관계형 데이터베이스 모델링

- **엔터티(Entity):** 정보를 기록하려는 항목 (예: 고객, 제품, 주문, 품목)
- **테이블(Table):** 실제 세계의 엔터티 컬렉션을 모델링
 - 고객 테이블: 고객 정보 저장
 - 제품 테이블: 제품 정의
 - 주문 테이블: 고객이 주문한 내역 저장
 - 품목 테이블: 주문에 포함된 개별 제품 저장
- **행(Row):** 테이블 내의 엔터티 인스턴스
 - 고객 테이블의 한 행 = 한 명의 고객 정보
 - 제품 테이블의 한 행 = 하나의 제품

관계형 테이블 특징

- 구조화된 데이터 형식
- 각 행은 동일한 열(Column) 구조를 가짐
- 일부 열은 선택적 값(NULL 가능)
 - 예: 고객 테이블의 MiddleName 열은 비어 있을 수 있음

데이터 형식 (Column Data Types)

- **문자형(Text):** 이메일, 이름 등
- **숫자형(Numeric):**
 - 정수(Integer) → 예: 주문 수량(Quantity)
 - 소수(Decimal) → 예: 제품 가격(Price)
- **날짜/시간(Date/Time):** 예: 주문 날짜(OrderDate)
- 사용 가능한 데이터 형식은 DBMS마다 다르지만, 대부분 ANSI 표준 데이터 형식을 지원

정규화를 이해하기

정규화 정의

- 데이터 중복 최소화, 데이터 무결성 적용을 위한 스키마 디자인 프로세스
- 여러 수준(정규형, Normal Form)이 있으나 실용적인 정의:
 1. 각 엔터티는 자체 테이블로 분리
 2. 각 불연속 특성은 자체 열로 분리
 3. 기본 키(Primary Key)로 각 행을 고유하게 식별
 4. 외래 키(Foreign Key)로 엔터티 간 연결

정규화 전

- 하나의 테이블(스프레드시트)에 고객·제품·주문 데이터가 모두 포함
- 고객 이름·주소, 제품 이름·가격이 반복적으로 기록 → **데이터 중복 발생**
- 구조화되지 않아 유지·관리 어려움

정규화 후

- **엔터티별 테이블** (고객, 제품, 주문, 품목)로 분리
- 각 엔터티의 불연속 특성은 **개별 열**에 저장
- 데이터 중복 제거 → 한 곳만 수정해도 전체에 반영 가능
- 열 분리로 데이터 형식 제한 가능 (예: 가격=10진수, 수량=정수)
- 쿼리에서 세분화된 분석 가능 (예: 특정 도시에 사는 고객 검색)
- **기본 키**로 행 식별, **외래 키**로 참조 무결성 보장

복합 키 (Composite Key)

- 여러 열의 고유 조합으로 키를 정의할 수 있음
- 예: Lineltem 테이블 = (OrderNo + ItemNo) 조합으로 품목을 고유 식별

SQL 살펴보기

SQL 개요

- SQL은 Structured Query Language(구조적 쿼리 언어)의 약자
- 관계형 데이터베이스 관리 시스템(RDBMS)의 **표준 언어**
- 주요 사용 목적: 데이터 검색, 업데이트, 삽입, 삭제 등
- 대표적인 RDBMS: Microsoft SQL Server, MySQL, PostgreSQL, MariaDB, Oracle
- SQL은 1986년 ANSI, 1987년 ISO에서 표준화됨

- 이후 벤더별로 확장 기능을 추가하여 다양한 SQL 버전이 존재

SQL 언어의 종류

- **Transact-SQL (T-SQL)**: Microsoft SQL Server, Azure SQL 서비스에서 사용
- **pgSQL**: PostgreSQL 전용 확장 포함
- **PL/SQL**: Oracle에서 사용 (Procedural Language/SQL)

SQL 문의(명령어) 3가지 그룹

1. DDL (데이터 정의 언어)

- 데이터베이스 개체(테이블, 뷰 등)를 생성/수정/삭제
- 주요 문법
 - CREATE: 새 개체 생성
 - ALTER: 구조 변경
 - DROP: 개체 삭제 (주의: 데이터 복구 불가)
 - RENAME: 개체 이름 변경

```
CREATE TABLE Product
(
  ID INT PRIMARY KEY,
  Name VARCHAR(20) NOT NULL,
  Price DECIMAL NULL
)
```

2. DCL (데이터 제어 언어)

- 사용자/그룹에 권한 부여, 거부, 회수
- 주요 문법
 - GRANT: 권한 부여
 - DENY: 권한 거부
 - REVOKE: 권한 회수

```
GRANT SELECT, INSERT, UPDATE
ON Product
TO user1
```

3. DML (데이터 조작 언어)

- 테이블의 행을 조회/삽입/수정/삭제

- 주요 문법
 - SELECT: 데이터 조회
 - INSERT: 새 행 삽입
 - UPDATE: 데이터 수정
 - DELETE: 행 삭제
- ⚠ 주의
 - WHERE 절을 생략하면 UPDATE/DELETE가 모든 행에 적용됨
 - DROP 문 실행 시 데이터 복구 불가

```
-- 조회
SELECT FirstName, LastName, Address, City
FROM Customer
WHERE City = 'Seattle'
ORDER BY LastName

-- JOIN
SELECT o.OrderNo, o.OrderDate, c.Address, c.City
FROM Order AS o
JOIN Customer AS c
ON o.Customer = c.ID

-- 수정
UPDATE Customer
SET Address = '123 High St'
WHERE ID = 1

-- 삭제
DELETE FROM Product
WHERE ID = 162

-- 삽입
INSERT INTO Product(ID, Name, Price)
VALUES (99, 'Drill', 4.99)
```

데이터베이스 개체 설명하기

뷰(View)

- SELECT 쿼리 결과를 기반으로 한 가상 테이블

- **특징**

- 실제 데이터를 복사하지 않고, 기본 테이블을 참조
- 테이블처럼 쿼리·필터링 가능

```
CREATE VIEW Deliveries AS
SELECT o.OrderNo, o.OrderDate,
       c.FirstName, c.LastName, c.Address, c.City
FROM Order AS o
JOIN Customer AS c
ON o.Customer = c.ID
```

저장 프로시저(Stored Procedure)

- 명령에 의해 실행되는 SQL 구문 집합
- **장점:**
 - 로직을 데이터베이스 내부에 캡슐화
 - 매개변수 활용 가능 → 재사용성 ↑

```
CREATE PROCEDURE RenameProduct
    @ProductID INT,
    @NewName VARCHAR(20)
AS
UPDATE Product
SET Name = @NewName
WHERE ID = @ProductID
```

```
-- 제품의 이름 변경
EXEC RenameProduct 201, 'Spanner'
```

인덱스(Index)

- 특정 열을 기준으로 정렬된 색인 구조 → 검색 속도 향상
- **특징**
 - 트리 구조 기반 탐색
 - 행이 많을수록 효과 ↑
 - 삽입·수정·삭제 시 인덱스 갱신 필요 → 추가 비용 발생
- **주의점:** 인덱스는 빠른 조회 vs. 쓰기 성능 저하 간 균형 필요

```
CREATE INDEX idx_ProductName  
ON Product(Name)
```